

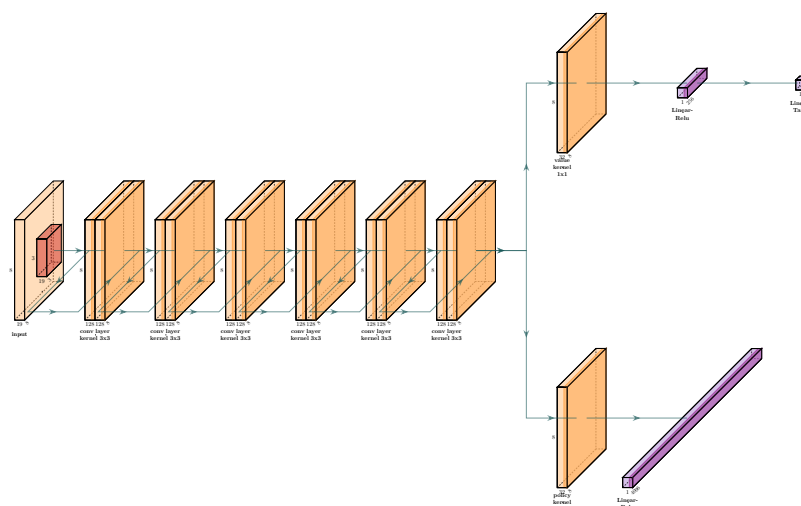
گزارش پروژه موتور شطرنج با یادگیری عمیق و جستجوی درخت مونت کارلو

۲ اسفند ۱۴۰۴

۱ مقدمه

هدف از این پروژه، طراحی و پیاده‌سازی یک موتور شطرنج با استفاده از یادگیری عمیق و الگوریتم جستجوی درخت مونت کارلو (MCTS) است. این موتور قادر است با استفاده از یک شبکه عصبی کانولوشن، موقعیت‌های صفحه شطرنج را ارزیابی کرده و با ترکیب آن با جستجوی MCTS، سعی کند تا بهترین حرکت را انتخاب کند. داده‌های آموزش از پایگاه داده Lichess دریافت و پس از فیلتر کردن بر اساس ریتینگ بازیکنان، برای آموزش شبکه عصبی استفاده می‌شوند.

۲ معماری شبکه عصبی



معماری شبکه

۱.۲ ورودی شبکه

ورودی شبکه یک تنسور $8 \times 8 \times 19$ است که هر لایه آن اطلاعات متفاوتی از صفحه شطرنج را نمایش می‌دهد:

- کانال‌های ۰-۵: موقعیت مهره‌های سفید (پیاده، اسب، فیل، رخ، وزیر، شاه)
- کانال‌های ۶-۱۱: موقعیت مهره‌های سیاه
- کانال‌های ۱۲-۱۵: حق قلعه رفتن
- کانال ۱۶: خانه آن پاسان
- کانال ۱۷: نوبت حرکت
- کانال ۱۸: تکرار موقعیت

۲.۲ لایه‌های شبکه

شبکه عصبی دارای ۶ بلاک اولیه Residual هست، که منظور از Residual این است که ورودی هر بلاک، با خروجی اش جمع خواهد شد [al. et He, ۲۰۱۵]. این سبب می‌شود که مشکل محو شدگی گرادینان ها برطرف شود. هر کدام از این بلاک ها شامل دو شبکه کانولوشنی Relu هستند، هر لایه کانولوشن از فیلترهای 3×3 با padding برابر ۱ استفاده می‌کند که باعث حفظ ابعاد 8×8 صفحه می‌شود. لایه‌های BatchNorm برای نرمال‌سازی استفاده شده‌اند. نتایج این شبکه سپس به دو سر سیاست (Policy Head) و سر ارزش (Value Head) داده می‌شود. که هدف سر سیاست، نگاه به صفحه شطرنج و تعیین احتمال هر حرکت در آن موقعیت است، و هدف سر ارزش، تعیین برتری بازیکن سفید یا سیاه است، که بین ۱- و ۱ قرار دارد. هر چه نزدیک تر به یک باشد، یعنی کاربر سفید برتری بیشتری دارد و هر چه نزدیک تر به ۱- باشد یعنی کاربر سیاه برتری بیشتری دارد. این معماری مشابه شبکه‌های استفاده شده در موتورهای معروفی مثل AlphaZero [Silver et al., 2017] است اما با ابعاد کوچک‌تر برای اجرا روی سخت‌افزار معمولی و برخلاف AlphaZero از Supervised Learning استفاده می‌شود. کد معماری شبکه در pytorch:

```
۱ class ResidualBlock(nn.Module):
۲     def __init__(self, channels):
۳         super().__init__()
۴
۵         self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
۶         self.bn1 = nn.BatchNorm2d(channels)
۷
۸         self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
۹         self.bn2 = nn.BatchNorm2d(channels)
۱۰
```

```

11     def forward(self, x):
12         identity = x
13
14         out = self.conv1(x)
15         out = self.bn1(out)
16         out = F.relu(out)
17
18         out = self.conv2(out)
19         out = self.bn2(out)
20
21         out = out + identity    # Skip connection
22         out = F.relu(out)
23
24     return out
25
26 class ChessNet(nn.Module):
27     def __init__(self, num_blocks=6):
28         super(ChessNet, self).__init__()
29
30         # Initial feature extraction
31         self.input_conv = nn.Sequential(
32             nn.Conv2d(19, 128, 3, padding=1),
33             nn.BatchNorm2d(128),
34             nn.ReLU(inplace=True),
35         )
36
37         # Residual tower
38         self.residual_layers = nn.Sequential(
39             *[ResidualBlock(128) for _ in range(num_blocks)]
40         )
41
42         # Policy Head
43         self.policy = nn.Sequential(
44             nn.Conv2d(128, 32, 1),
45             nn.BatchNorm2d(32),
46             nn.ReLU(inplace=True),
47             nn.Flatten(),
48             nn.Linear(32 * 8 * 8, 4096)
49         )
50
51         # Value Head
52         self.value = nn.Sequential(
53             nn.Conv2d(128, 32, 1),
54             nn.BatchNorm2d(32),

```

```

۵۵         nn.ReLU(inplace=True),
۵۶         nn.Flatten(),
۵۷         nn.Linear(32 * 8 * 8, 256),
۵۸         nn.ReLU(inplace=True),
۵۹         nn.Linear(256, 1),
۶۰         nn.Tanh()
۶۱     )
۶۲
۶۳     def forward(self, x):
۶۴         x = self.input_conv(x)
۶۵         x = self.residual_layers(x)
۶۶
۶۷         return self.policy(x), self.value(x)
۶۸ )

```

۳.۲ سر سیاست (Policy Head)

سر سیاست وظیفه تخمین احتمال تمامی حرکات در هر موقعیت شطرنج را بر عهده دارد:

```

۱ self.policy = nn.Sequential(
۲     nn.Conv2d(128, 32, 1),
۳     nn.BatchNorm2d(32),
۴     nn.ReLU(inplace=True),
۵     nn.Flatten(),
۶     nn.Linear(32 * 64, 4096)
۷ )

```

خروجی این سر یک بردار ۴۰۹۶ تایی است که هر عنصر نشان‌دهنده احتمال یک حرکت خاص است. تعداد ۴۰۹۶ از ضرب ۶۴ (خانه مبدأ) در ۶۴ (خانه مقصد) به دست آمده است. برخی از این حرکات ممکن است غیر قانونی باشند، به همین خاطر قبل از گرفتن softmax ابتدا حرکات غیر قانونی را منهای یک عدد منفی خیلی بزرگ می‌کنیم تا مقدار softmax آنها صفر و در نتیجه تأثیری در احتمال حرکات مجاز نداشته باشند.

۴.۲ سر ارزش (Value Head)

سر ارزش موقعیت را ارزیابی کرده و یک عدد بین -1 تا 1 برمی‌گرداند که نشان‌دهنده برتری سفید یا سیاه است:

```

۱ self.value = nn.Sequential(
۲     nn.Conv2d(128, 32, 1),
۳     nn.BatchNorm2d(32),

```

```

۴ nn.ReLU(inplace=True),
۵ nn.Flatten(),
۶ nn.Linear(32 * 64, 256),
۷ nn.ReLU(inplace=True),
۸ nn.Linear(256, 1),
۹ nn.Tanh()
۱۰ )

```

۳ ارزیابی هیوریستیک

برای برجسب‌زنی داده‌های آموزش بدون نیاز به موتورهای شطرنج مانند Stockfish، یک تابع ارزیابی هیوریستیک استفاده شده است که عوامل مختلفی را در نظر می‌گیرد:

۱.۳ ارزش مهره‌ها

```

۱ piece_values = {
۲     chess.PAWN: 100, chess.KNIGHT: 320, chess.BISHOP: 330,
۳     chess.ROOK: 500, chess.QUEEN: 900, chess.KING: 0
۴ }

```

۲.۳ جداول مکان مهره‌ها

برای هر مهره، یک جدول ۸×۸ از امتیازات موقعیتی تعریف شده است. به عنوان مثال، جدول پیاده‌ها به طور زیر است. که این یعنی هر چه پیاده به سطر آخر نزدیک تر باشد این بهتر است، چون رسیدن به سطر آخر به معنی ارتقا پیاده به مهره ای با ارزش بیشتر است. دلیل صفر بودن سطر آخر این است که وقتی پیاده به سطر آخر رسید، ارتقا خواهد یافت و دیگر پیاده نخواهد بود، پس هیچ وقت پیاده ای در سطر آخر وجود نخواهد داشت، پس مقادیر سطر آخر مهم نیست. برای بازیکن سیاه همین مقدار منفی خواهند شد و سطر هابرعکس می‌شوند.

```

۱ pawn_table = [
۲     [0, 0, 0, 0, 0, 0, 0, 0],
۳     [50, 50, 50, 50, 50, 50, 50, 50],
۴     [10, 10, 20, 30, 30, 20, 10, 10],
۵     [5, 5, 10, 25, 25, 10, 5, 5],
۶     [0, 0, 0, 20, 20, 0, 0, 0],
۷     [0, 0, 0, 0, 0, 0, 0, 0],
۸     [0, 0, 0, 0, 0, 0, 0, 0],
۹     [0, 0, 0, 0, 0, 0, 0, 0]
۱۰ ]

```

۳.۳ سایر عوامل

عوامل دیگری که در ارزیابی در نظر گرفته شده‌اند:

- تحرک مهره‌ها (تعداد حرکات قانونی)
- کنترل مرکز صفحه
- ایمنی شاه و حق قلعه رفتن
- سپر پیاده‌ای جلوی شاه

نهایتاً امتیاز نهایی با تابع $\tanh$ نرمال‌سازی می‌شود:

$$\text{value} = \tanh\left(\frac{\text{score}}{1000}\right)$$

۴ جستجوی درخت مونت کارلو (MCTS)

الگوریتم MCTS با ترکیب شبکه عصبی، جستجوی هوشمندانه‌ای برای یافتن بهترین حرکت انجام می‌دهد. این الگوریتم از ۴ مرحله تشکیل شده است، که این مراحل تکرار خواهند شد (در کد تعداد پیش فرض تکرار ۸۰۰ است).

۱.۴ مراحل MCTS

۱.۱.۴ انتخاب (Selection)

در این مرحله، با استفاده از فرمول PUTC، بهترین گره برای ادامه جستجو انتخاب می‌شود:

$$\text{score} = Q(s, a) + c \times P(s, a) \times \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

که در آن:

- $Q(s, a)$: برتری موقعیت شطرنج بعد از انجام حرکت a برای حرکت کننده.
- $P(s, a)$: احتمال اولیه حرکت از شبکه عصبی که توسط سر سیاست شبکه تعیین می‌شود
- $N(s)$: تعداد بازدیدهای گره والد
- $N(s, a)$: تعداد دفعاتی که این حرکت انتخاب شده
- c : ثابت اکتشاف (معمولاً 1.4) که تعیین می‌کند موتور شطرنج چقدر مشتاق به اکتشاف حرکات جدید است.

۲.۱.۴ بسط (Expansion)

اگر گره انتخاب شده قبلاً بسط داده نشده باشد، تمام حرکات قانونی به عنوان فرزندان آن ایجاد می‌شوند. احتمال اولیه هر حرکت از خروجی Head Policy شبکه عصبی گرفته می‌شود.

۳.۱.۴ ارزیابی (Evaluation)

اگر گره به حالت پایان بازی رسیده باشد، نتیجه واقعی بازی برگردانده می‌شود. در غیر این صورت، شبکه عصبی برای ارزیابی موقعیت فراخوانی می‌شود و خروجی سر ارزش به عنوان ارزش موقعیت در نظر گرفته می‌شود.

۴.۱.۴ بازگشت (Backup)

ارزش به دست آمده در تمام گره‌های مسیر جستجو به روزرسانی می‌شود:

- تعداد بازدیدها (*visits*) یک واحد افزایش می‌یابد
- مجموع ارزش‌ها (*value_sum*) با ارزش جدید جمع می‌شود
- برای گره‌های مربوط به حریف، علامت ارزش معکوس می‌شود

۲.۴ پیاده‌سازی MCTS

کلاس‌های اصلی MCTS به صورت زیر پیاده‌سازی شده‌اند:

```
۱ class MCTSNode:
۲     def __init__(self, board, parent=None, move=None, prior=0):
۳         self.board = board
۴         self.parent = parent
۵         self.move = move
۶         self.prior = prior
۷         self.visits = 0
۸         self.value_sum = 0
۹         self.children = {}
۱0        self.is_expanded = False
۱۱
۱۲    def value(self):
۱۳        if self.visits == 0:
۱۴            return 0
۱۵        return self.value_sum / self.visits
۱۶
۱۷    def ucb_score(self, exploration_constant=1.4):
۱۸        if self.visits == 0:
```

```

19         return float('inf')
20     exploration = exploration_constant * self.prior * \
21         math.sqrt(self.parent.visits) / (1 + self.visits)
22     return self.value() + exploration

```

و کلاس اصلی MCTS:

```

1  class MCTS:
2      def __init__(self, model, num_simulations=800):
3          self.model = model
4          self.num_simulations = num_simulations
5          self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
6
7      def search(self, board):
8          root = MCTSNode(board.copy())
9          for _ in range(self.num_simulations):
10             node = root
11             path = [node]
12             # Selection
13             while node.is_expanded and node.children:
14                 node = max(node.children.values(), key=lambda c: c.ucb_score())
15                 path.append(node)
16
17             # Expansion
18             if not node.board.is_game_over() and not node.is_expanded:
19                 self.expand(node)
20                 node.is_expanded = True
21
22             # Evaluation
23             if node.board.is_game_over():
24                 # Game over
25                 result = node.board.result()
26                 if result == "1-0":
27                     value = 1
28                 elif result == "0-1":
29                     value = -1
30                 else:
31                     value = 0.0
32             else:
33                 # Use neural network for evaluation
34                 value = self.evaluate(node.board)
35             if node.board.turn == chess.WHITE:
36                 value = -value

```

```

۳۷         #Backpropagation
۳۸         for node in reversed(path):
۳۹             node.visits += 1
۴۰             node.value_sum += value
۴۱             value = -value # Flip for next parent
۴۲
۴۳     return root

```

۵ آماده‌سازی داده‌ها

داده‌های آموزش از پایگاه داده Lichess دریافت می‌شوند. این پایگاه داده شامل میلیون‌ها بازی شطرنج با فرمت PGN یا CVS است.

۱.۵ دریافت و فیلتر کردن فایل‌ها

فایل‌های PGN و CSV با فرمت فشرده zst از آدرس <https://database.lichess.org/> دریافت می‌شوند. در این پروژه از سه فایل استفاده شده است:

- lichess_db_standard_rated_2015-01.pgn.zst
- lichess_db_standard_rated_2013-01.pgn.zst
- lichess_db_puzzle.csv.zst

برای فیلتر کردن بر اساس ریتینگ، تابع `safe_elo` نوشته شده است. دو فایل اول شامل بازی‌های مختلف بین افراد مختلف است، فایل سوم شامل یکسری پازل شطرنج، همراه با راه حل آن‌هاست. ۸۰ درصد این داده‌های برای `train` مدل و ۲۰ درصد برای `validation` به کار رفته است.

۲.۵ استخراج موقعیت‌ها

از هر بازی، تعدادی موقعیت تصادفی (پیش‌فرض ۱۵ موقعیت) استخراج می‌شود. برای هر موقعیت، موارد زیر ذخیره می‌گردد:

- تنسور صفحه شطرنج (۸×۸×۱۹)
- ارزش موقعیت از تابع ارزیابی هیوریستیک
- حرکتی که در آن موقعیت انجام شده که با عنوان هدف `policy` از آن استفاده می‌شود.

۶ آموزش مدل

فرآیند آموزش با استفاده از بهینه‌ساز AdamW و تابع `Loss` ترکیبی انجام می‌شود.

۱.۶ تابع Loss

تابع Loss از دو بخش تشکیل شده است:

$$\mathcal{L} = \mathcal{L}_{\text{value}} + \mathcal{L}_{\text{policy}}$$

که در آن:

- $\mathcal{L}_{\text{value}} = \text{MSE}(v_{\text{pred}}, v_{\text{target}})$ خطای میانگین مربعات برای سر ارزش
- $\mathcal{L}_{\text{policy}} = \text{CrossEntropy}(p_{\text{pred}}, p_{\text{target}})$ آنتروپی متقاطع برای سر سیاست

۲.۶ جلوگیری از بیش‌برازش

برای جلوگیری از بیش‌برازش (Overfitting)، چندین روش استفاده شده است:

۱.۲.۶ Decay Weight

در بهینه‌ساز AdamW از پارامتر `weight_decay` با مقدار $1e-4$ استفاده شده است. این کار یک جریمه به وزن‌های بزرگ اضافه می‌کند و باعث ساده‌تر شدن مدل می‌شود.

۲.۲.۶ Stopping Early

در هر epoch بهترین مدل بر اساس کمترین خطای اعتبارسنجی انتخاب می‌گردد و ذخیره می‌شود:

```
۱ if avg_val < best_val_loss:
۲     best_val_loss = avg_val
۳     torch.save(self.model.state_dict(),
۴                 f"chess_model_BEST_val_{best_val_loss:.4f}.pth")
```

۷ اجرای برنامه

ابتدا پکیج‌های مورد نیاز برنامه به طور زیر نصب می‌شوند:

```
۱ python -m pip install -r requirements.txt
```

سپس مدل را با دستور زیر می‌شود اجرا کرد:

```
۱ python ./chess_engine_main.py
```

برنامه دارای یک منوی تعاملی با گزینه‌های زیر است:

Options:

1. Train model
2. Load existing model
3. Play against engine
4. Watch engine vs random
5. Watch engine self-play
6. Exit

که اول گزینه دو برای لود کردن مدل تمرین داده شده و سپس می‌شود با مدل بازی کرد. برای تمرین دادن مدلی جدید باید ابتدا دستور:

```
1 python ./data.py
```

اجرا شود، این فایل داده‌های مورد نیاز برای تمرین مدل را آماده و در پوشه: chess_data ذخیره می‌کند. سپس با اجرای فایل chess_engine_main.py و انتخاب گزینه ۱ برای تمرین مدل شروع می‌شود و مدل‌های یادگیری شده در فایل‌هایی با پسوند pth ذخیره می‌شوند که آن‌ها را میتوان با استفاده از گزینه ۲ لود کرد.

۱.۷ آماده‌سازی داده‌ها

برای آماده‌سازی داده‌ها، اسکریپت data.py اجرا می‌شود. این اسکریپت فایل‌های zst را پردازش کرده و فایل‌های دسته‌ای با نام chess_data_batch_*.pt را در ایجاد میکند، که برای یادگیری استفاده می‌شوند.

۲.۷ آموزش مدل

پس از آماده‌سازی داده‌ها، با انتخاب گزینه ۱ از منو و وارد کردن پارامترهای آموزش، فرآیند آموزش آغاز می‌شود. در طول آموزش، مدل‌ها با نام‌هایی مانند زیر ذخیره می‌شوند:

• chess_model_BEST_val_2.0604.pth

• chess_model_BEST_val_2.0529.pth (بهترین مدل تا الان)

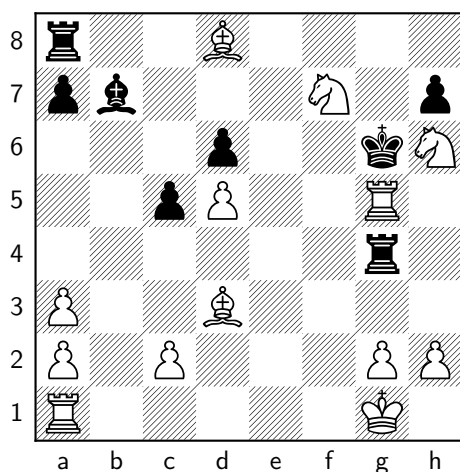
۳.۷ بازی با موتور

با انتخاب گزینه‌های ۳ تا ۵ می‌توان با موتور بازی کرد یا آن را در مقابل حرکات تصادفی مشاهده نمود. موتور با انجام ۸۰۰ شبیه‌سازی MCTS برای هر حرکت، بهترین حرکت را انتخاب می‌کند.

۴.۷ نتایج

این مدل با استفاده از 11.5 میلیون موقعیت بازی شطرنج برای 14 epoch یادگیری داده شده. در هنگام بازی، تعداد شبیه سازی های MCTS برابر ۸۰۰ قرار داده شده. در ادامه برخی بازی های این موتور در برابر حرکات شانسی نمایش داده شده (سفید موتور است و سیاه حرکات شانسی):

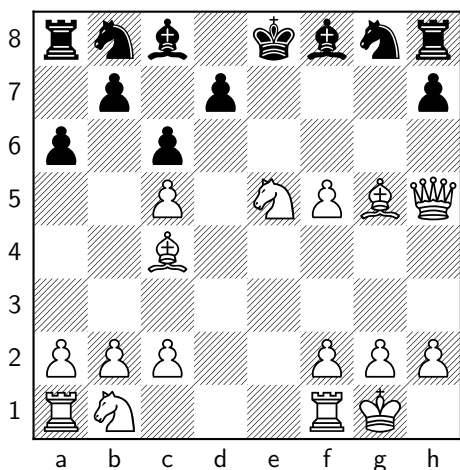
1 e4 ♘c6 2 ♗f3 e6 3 d4 ♗h6 4 ♗c3 ♙a3 5 bxa3 ♚f6 6 ♙d3 ♚d8
7 O-O ♗e5 8 ♗xe5 c5 9 f4 g5 10 ♚h5 b5 11 ♚xf7+ ♗xf7 12 ♗b5
♜g8 13 d5 gxf4 14 ♙xf4 exd5 15 exd5 ♙b7 16 ♗d6+ ♔e7 17 ♗f5+
♔f6 18 ♗xf7 ♜g4 19 ♙g5+ ♔g6 20 ♗f5h6 ♔h5 21 ♙xd8 d6 22 ♜f5
♔g6 23 ♜g5#



white wins

بازی دوم:

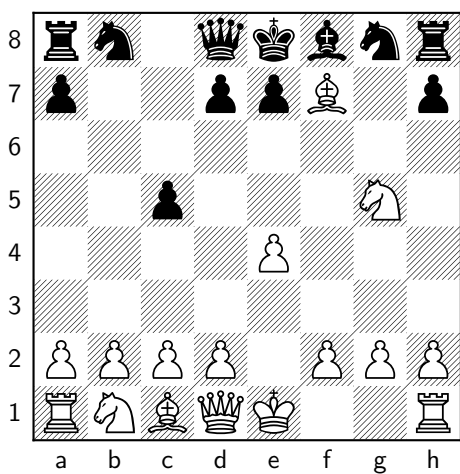
1 e4 g5 2 d4 a6 3 ♗f3 c6 4 ♙c4 ♚b6 5 O-O ♚c5 6 dxc5 e5 7
♙g5 f5 8 exf5 ♜a7 9 ♗e5 ♜a8 10 ♚h5



white wins

بازی سوم:

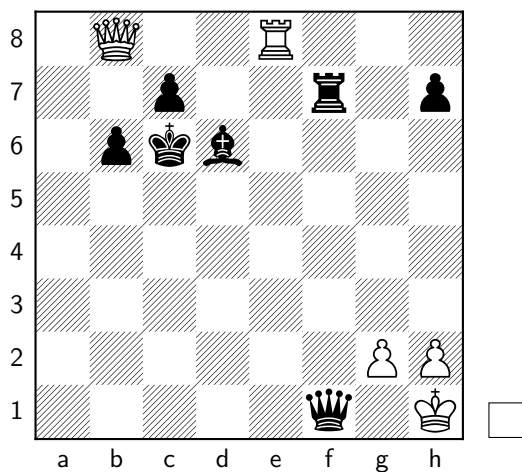
1 e4 ♘h6 2 ♘f3 b5 3 ♙xb5 g5 4 ♘xg5 ♙a6 5 ♙xa6 ♘g8 6 ♙c4 c5 7 ♙xf7#



white wins

در ادامه بازی ای داریم که موتور در برابر خودش بازی کرده:

1 e4 ♘f6 2 e5 ♘d5 3 ♘f3 ♘c6 4 ♙c4 ♘b6 5 ♙b3 d5 6 exd6 ♙xd6 7 O-O ♙g4 8 d4 ♙xf3 9 ♙xf3 ♘xd4 10 ♙xf7+ ♙d8 11 ♘c3 ♘xb3 12 axb3 ♙d7 13 ♙e3 ♙c6 14 ♙ad1+ ♙c8 15 ♙xb6 axb6 16 ♙fe1 e6 17 ♘e4 ♙a5 18 ♙d3 ♙e5 19 ♙f4 ♙xe4 20 ♙xe4 ♙xc2 21 ♙xe6 ♙xd3 22 ♙e8+ ♙d7 23 ♙e1 ♙d6 24 ♙f7+ ♙c6 25 ♙xg7 ♙d8 26 ♙f7 ♙d7 27 ♙e8 ♙xb3 28 ♙e4+ ♙b5 29 ♙xb7 ♙xb2 30 ♙c8 ♙c6 31 ♙e8 ♙f7 32 ♙b8 ♙xf2+ 33 ♙h1 ♙f1#



black wins

References

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. 2015 URL <https://arxiv.org/abs/1512.03385>.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. 2017 URL <https://arxiv.org/abs/1712.01815>.